



---

# Rapport Intermédiaire

Auteurs : **Yoann Rabahi / Emmanuel Santos**

Pour le 25 Juin 2026

---

Effectué à

**L'EFREI**

Module

**Programmation avancée en JAVA**

En vue de l'obtention du

**Titre d'ingénieur**

**Parcours** : Big Data & Machine Learning (BDML)

Sous l'encadrement

**Académique** : NAHLE Nada

---

*Année académique : 2025 – 2026*

# Table des matières

|          |                                                              |          |
|----------|--------------------------------------------------------------|----------|
| <b>1</b> | <b>Présentation du projet</b>                                | <b>1</b> |
| <b>2</b> | <b>Modèle métier</b>                                         | <b>1</b> |
| 2.1      | Diagramme de classes UML . . . . .                           | 1        |
| <b>3</b> | <b>Persistance des données</b>                               | <b>2</b> |
| 3.1      | Schéma de la base de données . . . . .                       | 2        |
| 3.2      | Sauvegarde et hydratation des objets Java . . . . .          | 2        |
| <b>4</b> | <b>Architecture</b>                                          | <b>3</b> |
| 4.1      | Les différentes couches et classes . . . . .                 | 3        |
| 4.2      | Fichiers FXML utilisés . . . . .                             | 3        |
| <b>5</b> | <b>Principes SOLID</b>                                       | <b>4</b> |
| <b>6</b> | <b>Captures d'écran</b>                                      | <b>4</b> |
| 6.1      | Gestion des histoires . . . . .                              | 4        |
| 6.2      | Gestion des personnages . . . . .                            | 4        |
| 6.3      | Gestion des scènes . . . . .                                 | 5        |
| 6.4      | Affichage des statistiques . . . . .                         | 6        |
| 6.5      | principales fonctionnalités réalisées . . . . .              | 6        |
| <b>7</b> | <b>Tests réalisés</b>                                        | <b>7</b> |
| 7.1      | Tests Unitaires : Validation de la logique métier . . . . .  | 7        |
| 7.2      | Tests d'Intégration : Robustesse de la persistance . . . . . | 7        |
| 7.3      | Bilan de l'exécution . . . . .                               | 8        |
| <b>8</b> | <b>Difficultés rencontrées et solutions apportées</b>        | <b>8</b> |
| <b>9</b> | <b>Perspectives d'évolution du projet</b>                    | <b>9</b> |

## Table des figures

|   |                                                                        |   |
|---|------------------------------------------------------------------------|---|
| 1 | Modèle Conceptuel des Données . . . . .                                | 2 |
| 2 | Interface de gestion et de sélection des histoires . . . . .           | 4 |
| 3 | Formulaire de création et de suivi des personnages (Casting) . . . . . | 5 |
| 4 | Espace de travail des scènes et attribution du casting . . . . .       | 5 |
| 5 | Tableau de bord des statistiques . . . . .                             | 6 |

# 1 Présentation du projet

L'application **Story Forge** est un outil d'aide à la création littéraire et à la scénarisation. Son objectif principal est de permettre à un auteur de structurer ses récits, de gérer son univers de manière cohérente et de suivre l'évolution de ses intrigues à travers une interface graphique intuitive.

Les principales fonctionnalités réalisées et opérationnelles au sein de cette version intermédiaire sont :

- **La gestion des histoires** : Création, consultation, modification et suppression des projets de récits globaux.
- **La gestion du casting (Personnages)** : Déclaration des personnages, attribution d'un rôle (protagoniste, antagoniste, secondaire) et d'une description textuelle.
- **La gestion de la structure narrative (Scènes)** : Découpage de l'histoire en scènes distinctes caractérisées par un lieu, un moment chronologique, une position d'ordre unique et un statut d'avancement.
- **La mise en relation des entités** : Association dynamique et désassociation (relation *Many-To-Many*) des personnages présents au sein d'une scène spécifique.

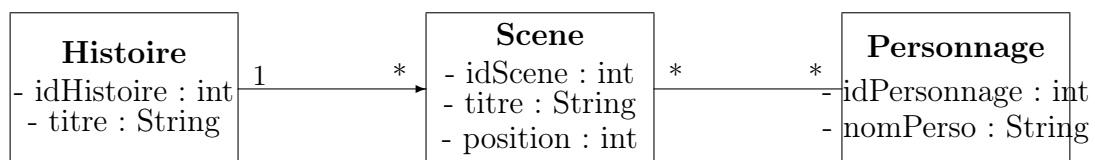
## 2 Modèle métier

Le modèle de Story Forge repose sur trois entités :

- **Histoire** : Représente le projet global. Elle centralise une liste de scènes et une liste de personnages.
- **Personnage** : Contient l'identité, le rôle et la description d'un acteur du récit.
- **Scene** : Représente l'unité d'action spatio-temporelle du récit. Elle est rattachée à une unique histoire (*Many-To-One*) et inclut une liste de personnages présents (*Many-To-Many*).

### 2.1 Diagramme de classes UML

Le diagramme ci-dessous illustre l'organisation de nos entités ainsi que les multiplicités de leurs relations :



*Schématisation conceptuelle du modèle métier (One-To-Many et Many-To-Many)*

## 3 Persistance des données

### 3.1 Schéma de la base de données

La persistance est assurée par un système de gestion de base de données relationnelle MySQL. Le schéma comprend trois tables principales et une table de jointure :

TABLE 1 – Structure relationnelle des tables SQL

| Table            | Clé Primaire    | Clés Étrangères / Liaisons                        |
|------------------|-----------------|---------------------------------------------------|
| histoire         | id_histoire     | Aucune                                            |
| personnage       | id_personnage   | id_histoire (Clé étrangère, relation 1-N)         |
| scene            | id_scene        | id_histoire (Clé étrangère, relation 1-N)         |
| scene_personnage | Co-clé primaire | id_scene, id_personnage (Table d'association N-N) |

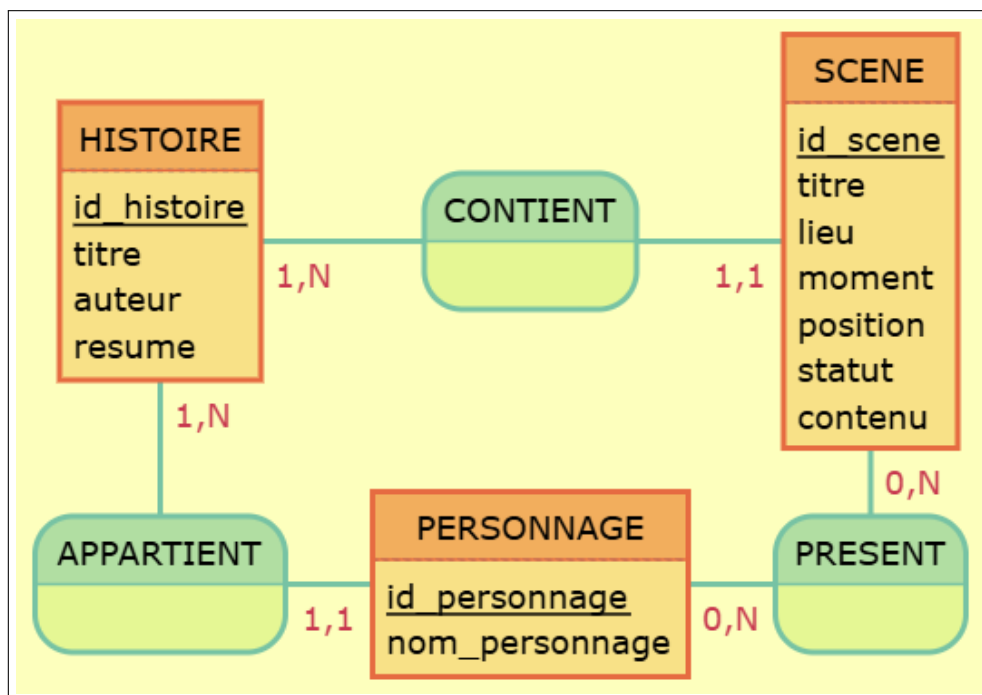


FIGURE 1 – Modèle Conceptuel des Données

### 3.2 Sauvegarde et hydratation des objets Java

Pour sauvegarder et charger les objets Java sans utiliser d'ORM lourd, l'application s'appuie sur le patron de conception **Data Access Object (DAO)** via l'API JDBC. Les requêtes SQL (ex : INSERT, UPDATE, SELECT) sont préparées à l'aide de `PreparedStatement` pour éviter les injections SQL. L'hydratation des instances s'effectue en lisant le `ResultSet` ligne par ligne, puis en appliquant les valeurs lues aux instances à l'aide des constructeurs par défaut combinés aux *setters* appropriés, assurant l'indépendance de la mémoire par rapport au modèle physique.

## 4 Architecture

Le projet adopte une architecture en couches découplées pour isoler la logique de présentation, la logique métier et l'accès aux données.

### 4.1 Les différentes couches et classes

- **Couche Modèle (model)** : Contient les objets métiers (*Histoire*, *Personnage*, *Scene*) exempts de toute logique technique ou SQL.
- **Couche Accès aux données (repository & dao)** : La classe *DatabaseConnection* centralise l'ouverture de la session JDBC. La classe *MySQLSceneRepository* implémente les opérations CRUD brutes et l'extraction des jointures d'association.
- **Couche Métier (service)** : La classe *SceneService* orchestre la logique applicative. Elle valide les contraintes métier (champs obligatoires non vides, unicité stricte de la position d'une scène au sein d'une même histoire) avant de déléguer la persistance au dépôt.
- **Couche Présentation (controller)** : Les contrôleurs JavaFX (*MainController*, *HistoireController*, *SceneController*) capturent les actions de l'utilisateur, mettent à jour les vues graphiques et délèguent les traitements complexes à la couche Service. Le contrôleur *DashboardController* permet d'afficher différentes statistiques concernant les histoires.

### 4.2 Fichiers FXML utilisés

L'interface graphique est modulaire et segmentée en fichiers FXML distincts imbriqués dans une vue racine :

- *hello-view.fxml* : Conteneur principal de l'application gérant la disposition générale de la fenêtre, l'agencement des volets et l'inclusion des sous-vues (mise en page globale).
- *histoire-view.fxml* : Gère le panneau latéral de gauche dédié à la liste des histoires, permettant la sélection du projet actif et le chargement dynamique de ses données.
- *scene-view.fxml* : Gère l'interface centrale et inférieure dédiée aux scènes narratives (liste des scènes, formulaires de description et composants d'association du casting).
- *personnage-view.fxml* : Gère le volet ou formulaire dédié au casting de l'histoire, permettant la déclaration, la modification et le listing des personnages.
- *dashboard-view.fxml* : Complètement indépendant de la phase d'édition, ce panneau est dédié à la restitution visuelle des indicateurs de performance du projet (statistiques globales, répartition des scènes par statut, volumes de personnages).

## 5 Principes SOLID

Au moins deux principes SOLID ont été rigoureusement appliqués au sein de la réarchitecture :

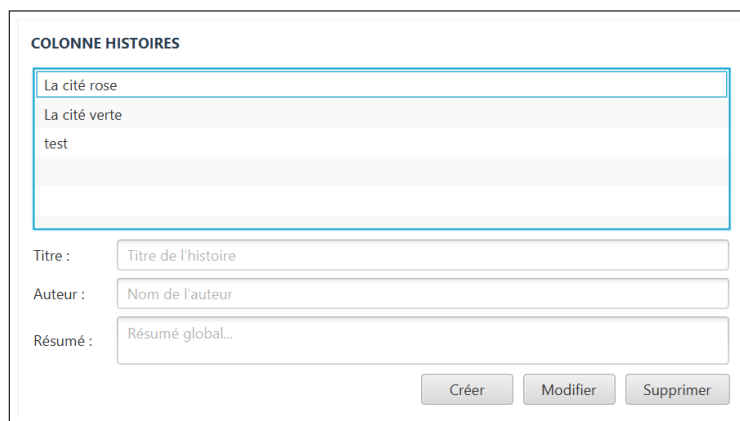
1. **Single Responsibility Principle (SRP) – Principe de responsabilité unique :** Chaque classe n'a qu'une seule raison de changer. Par exemple, `SceneController` est uniquement responsable de l'affichage et de l'interception graphique ; il ne contient aucune requête SQL ni aucune règle de validation. Ces responsabilités sont déléguées respectivement à `MySQLSceneRepository` et à `SceneService`.
2. **Dependency Inversion Principle (DIP) – Principe d'inversion des dépendances :** Les modules de haut niveau ne dépendent pas directement des modules de bas niveau. Le `SceneService` reçoit son instance de `MySQLSceneRepository` par injection via son constructeur, ce qui permettrait de remplacer le stockage MySQL par un stockage en mémoire ou sur fichier sans impacter la logique de validation de l'application.

## 6 Captures d'écran

Cette section présente les interfaces graphiques de l'application Story Forge, mettant en évidence l'intégration visuelle des fonctionnalités requises.

### 6.1 Gestion des histoires

La figure 2 présente l'interface principale d'accueil. Le panneau latéral gauche liste l'ensemble des projets d'écriture extraits de la base de données. La sélection d'une histoire charge dynamiquement ses données associées, tandis que le formulaire dédié permet d'éditer ou d'ajouter de nouveaux récits de manière centralisée.



COLONNE HISTOIRES

La cité rose  
La cité verte  
test

Titre : Titre de l'histoire  
Auteur : Nom de l'auteur  
Résumé : Résumé global...

Créer Modifier Supprimer

FIGURE 2 – Interface de gestion et de sélection des histoires

### 6.2 Gestion des personnages

La figure 3 illustre le module de configuration du casting. Cet écran permet de lister les personnages rattachés à l'histoire active, de définir explicitement leurs profils (nom, rôle narratif et description) et de persister ces informations au sein de la table `personnage`.

**ZONE PERSONNAGES**

**Casting (Sélectionnez une histoire) :** Choisir une histoire...

Nom :

Rôle :

Description :

Ajouter au casting Modifier Supprimer

FIGURE 3 – Formulaire de création et de suivi des personnages (Casting)

### 6.3 Gestion des scènes

La figure 4 expose l'interface de structuration narrative. Elle affiche la liste ordonnée des scènes (triées par position) et leurs informations relatives (lieu, moment, statut). La section de droite met en évidence la résolution de la table associative, affichant en temps réel les personnages présents au sein de la scène sélectionnée.

**Détails de la scène sélectionnée**

Statut

Ajouter Modifier Supprimer

FIGURE 4 – Espace de travail des scènes et attribution du casting

## 6.4 Affichage des statistiques

La figure 5 met en valeur les différents indicateurs liés aux scènes, notamment le nombre de personnages par scènes ainsi que les différents statut de chacune.

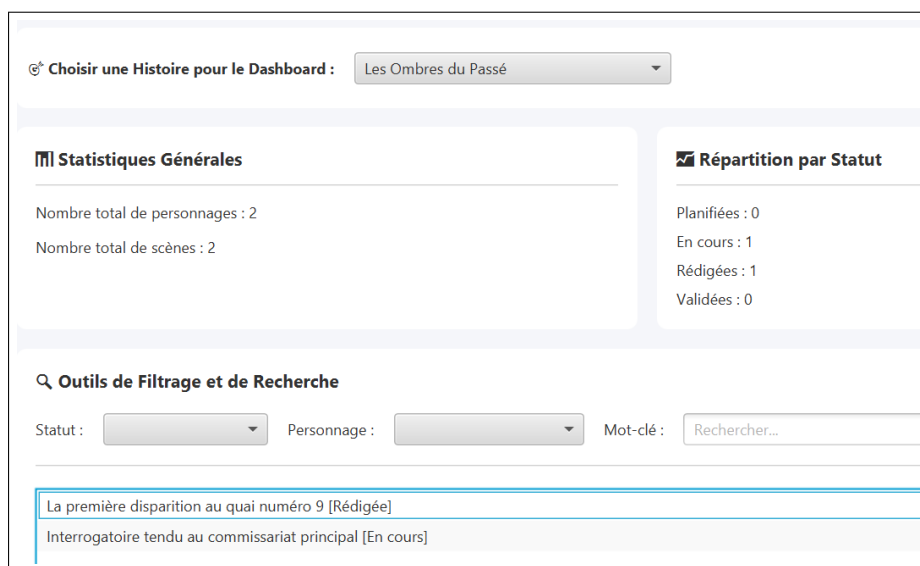


FIGURE 5 – Tableau de bord des statistiques

## 6.5 principales fonctionnalités réalisées

En conclusion, cette phase de développement intermédiaire a permis de finaliser une interface graphique moderne, fluide et entièrement synchronisée avec notre base de données. L'utilisateur dispose d'un espace de travail ergonomique et modulaire pour piloter ses projets d'écriture.

En résumé, les principales fonctionnalités visibles et opérationnelles au sein de l'interface sont :

- **Navigation multi-projets (Panneau Histoires) :** Sélection instantanée d'un projet dans la liste latérale gauche, entraînant le rafraîchissement global et automatique de toutes les autres vues de l'écran.
- **Formulaires de saisie dynamiques :** Formulaires complets avec champs textuels (*TextField*), zones de texte enrichies (*TextArea*) pour le contenu des scènes, et menus déroulants (*ComboBox*) pour la gestion des statuts de progression.
- **Suivi et ordonnancement visuel :** Affichage des scènes sous forme de liste triée chronologiquement selon leur position, offrant une vue d'ensemble claire de la structure du récit.
- **Système d'alertes intégré :** Pop-ups contextuelles bloquantes avertissant immédiatement l'utilisateur en cas d'erreur de saisie (par exemple, la saisie d'un texte à la place d'un numéro de position) ou de doublon.
- **Contrôle du casting en temps réel (Gestion N-N) :** Zone d'affectation interactive permettant d'ajouter un personnage à une scène via un menu déroulant, ou de l'exclure d'un simple clic, avec mise à jour visuelle instantanée de la liste des personnages présents.

## 7 Tests réalisés

La validation de l'application *Story Forge* repose sur une stratégie de tests automatisés rigoureuse, articulée autour du framework **JUnit 5**. Afin de garantir la robustesse globale du logiciel, la couverture de code a été divisée en deux grandes catégories complémentaires : les tests unitaires et les tests d'intégration. Cette double approche assure la conformité de l'application avec le scénario fonctionnel de démonstration imposé.

### 7.1 Tests Unitaires : Validation de la logique métier

Les tests unitaires ont pour objectif d'isoler et de valider les règles de gestion de l'application, indépendamment de toute infrastructure réseau ou logicielle. Pour ce faire, les couches d'accès aux données ont été simulées à l'aide de *Stubs* (bouchons en mémoire basés sur des `ArrayList`), évitant ainsi le recours à une base de données active.

La classe `MetierTest` exécute une série de vérifications :

- **Intégrité des données d'entrée** : Validation du rejet des chaînes vides ou `null` lors de la création d'une histoire ou d'une scène, levant systématiquement une exception `IllegalArgumentException`.
- **Contraintes d'unicité du casting** : Vérification de l'impossibilité d'ajouter deux personnages possédant le même nom au sein d'une même histoire. Le test valide également que cette comparaison est insensible à la casse (ex. : la détection du doublon entre « Gandalf » et « GANDALF »).
- **Cohérence des associations** : Blocage algorithmique de la tentative d'associer un personnage à une scène si ce dernier ne fait pas partie de l'histoire active.
- **Pertinence du moteur de recherche et de filtrage** : Validation des algorithmes de filtrage par statut (ex. : « En cours », « Prêt à publier »), par personnage présent, ainsi que de la recherche textuelle floue (insensible à la casse) dans le titre ou le contenu des scènes.

### 7.2 Tests d'Intégration : Robustesse de la persistance

Les tests d'intégration valident la communication et la cohabitation entre le code Java, le driver JDBC et le modèle relationnel de données. Afin de garantir l'indépendance de ces tests et d'éviter toute altération des données de production, une base de données **H2 en mémoire éphémère** a été implémentée (`DatabaseConnectionTest`). Le schéma SQL y est injecté à l'initialisation, puis les tables sont intégralement purgées entre chaque test (`clearAll()`).

La classe `PersistanceTest` simule des interactions réelles avec la base de données :

- **Cycle de vie des entités** : Validation de la bonne insertion, de la mise à jour et de la relecture complète des objets `Personnage` et `Scene` avec leurs attributs spécifiques (lieu, moment, position, statut).
- **Validation de la suppression** : Vérification stricte de la suppression effective d'un enregistrement en base et de la non-persistance de ses données après l'opération.

- **Intégrité référentielle en cascade** : Validation du comportement des clés étrangères SQL. Le test confirme que la suppression d'une entité `Histoire` entraîne automatiquement, par effet de cascade (`ON DELETE CASCADE`), la destruction de l'ensemble de ses scènes, personnages et liaisons associés dans la table `scene_personnage`.

### 7.3 Bilan de l'exécution

L'ensemble des suites de tests s'exécute avec succès en environnement d'intégration continue (ou au sein de l'IDE). Le recours aux bases en mémoire pour l'intégration et aux *stubs* pour le test unitaire permet d'obtenir un temps d'exécution global inférieur à deux secondes.

## 8 Difficultés rencontrées et solutions apportées

Le développement de l'application *Story Forge* a présenté plusieurs défis techniques majeurs, principalement liés à l'intégration des technologies choisies et à la synchronisation des données.

- 1. Problématique de la duplication de classes sous l'IDE IntelliJ :**
  - *Difficulté* : Lors de la restructuration de l'architecture modulaire, des conflits de nommage et des erreurs de duplication de classes (notamment sur le `MainController`) sont apparus, bloquant la compilation du projet.
  - *Solution* : Une vérification stricte de la portée des packages et le nettoyage complet des fichiers compilés (*Build* → *Clean* / *Rebuild Project*), suivis d'une invalidation des caches de l'IDE, ont permis de réinitialiser l'état du compilateur et de résoudre le conflit.
- 2. Contraintes d'intégrité de la base de données (MySQL Error 1364) :**
  - *Difficulté* : Lors de la phase de test avec l'insertion de jeux de données, les requêtes `INSERT` échouaient en raison de contraintes `NOT NULL` strictes sur des champs spécifiques (comme `contenu` ou `position` dans la table `scene`) n'ayant pas de valeurs par défaut configurées.
  - *Solution* : Les scripts d'insertion ont été corrigés pour intégrer l'ensemble des attributs obligatoires requis par le schéma relationnel, garantissant la cohérence fonctionnelle du Dashboard dès le démarrage.
- 3. Synchronisation de l'affichage dynamique multi-vues (JavaFX) :**
  - *Difficulté* : L'architecture modulaire reposant sur plusieurs fichiers FXML (*histoire-view*, *scene-view*, etc.) posait un problème de persistance visuelle. Initialement, la sélection d'une scène ou d'un personnage ne déclenchait pas la mise à jour des éléments associés à l'écran, ou provoquait des affichages bruts sous forme d'adresses mémoire.
  - *Solution* : L'implémentation de `CellFactory` personnalisées sur les composants `ListView` et `ComboBox` a permis de formater proprement les données textuelles. De plus, l'injection de requêtes de jointure SQL directement lors du rafraîchissement des scènes (via la table de liaison `scene_personnage`) a assuré une synchronisation parfaite du casting en temps réel.

## 9 Perspectives d'évolution du projet

Bien que le cœur fonctionnel de l'application soit opérationnel et valide les exigences du cycle de développement, plusieurs axes d'amélioration peuvent être envisagés pour transformer ce prototype en une solution logicielle industrielle :

- **Migration vers une architecture hybride ou déconnectée (SQLite) :** Une évolution majeure consisterait à intégrer un mécanisme de base de données embarquée via **SQLite**. Cela permettrait aux utilisateurs de travailler localement en mode hors-ligne sans dépendre d'un serveur MySQL distant, avec un système de synchronisation automatique lors de la reconnexion.
- **Amélioration de l'IHM et UX :** L'ajout de drag-and-drop pour réordonner visuellement la position des scènes au sein d'une histoire, ainsi que l'intégration de graphiques plus avancés (via **BarChart** ou **PieChart** de JavaFX) dans le volet **dashboard-view**, enrichiraient considérablement l'expérience utilisateur.
- **Système de versioning des récits :** Permettre la création de branches narratives ou de sauvegardes historiques (versions antérieures d'une scène ou d'un résumé) pour offrir aux auteurs un véritable outil de gestion de configuration de leurs œuvres.